

VIDEO COURSE

Containers Module - Video Course Outline

Course Title: Mastering Container Orchestration with Go

Duration: ~4 hours

Level: Intermediate to Advanced

Prerequisites: Go basics, container fundamentals

Module 1: Introduction and Setup (20 minutes)

Lesson 1.1: Course Overview

- What you'll learn
- Prerequisites checklist
- Course structure

Lesson 1.2: Module Installation

```
go get digital.vasic.containers@latest
```

- Go module setup
- Dependency management
- Development environment

Lesson 1.3: Quick Demo

- Auto-detecting container runtime
 - Starting a compose project
 - Health checking services
-

Module 2: Container Runtimes (30 minutes)

Lesson 2.1: Runtime Abstraction

- Why abstraction matters
- ContainerRuntime interface
- Supported runtimes

Lesson 2.2: Auto-Detection

```
rt, err := runtime.AutoDetect(ctx)
```

- Detection order: Docker → Podman → Kubernetes
- Manual runtime selection
- Runtime capabilities

Lesson 2.3: Basic Operations

- List containers
- Inspect containers
- Start/Stop/Remove
- Exec into containers

Lab 2: Working with Runtimes

- Detect your local runtime
 - List all running containers
 - Execute commands in containers
-

Module 3: Compose Orchestration (40 minutes)

Lesson 3.1: ComposeOrchestrator Interface

- Up, Down, Status, Logs
- ComposeProject structure
- Profiles and services

Lesson 3.2: Starting Services

```
project := compose.ComposeProject{  
    File: "docker-compose.yml",  
    Name: "myapp",  
}
```

```
}  
orch.Up(ctx, project)
```

Lesson 3.3: Managing Services

- Checking service status
- Viewing logs
- Stopping services

Lab 3: Compose Management

- Create a compose file
 - Start services programmatically
 - Monitor and stop services
-

Module 4: Health Checking (30 minutes)

Lesson 4.1: Health Check Strategies

- TCP health checks
- HTTP health checks
- gRPC health checks
- Custom health checks

Lesson 4.2: Retry Policies

```
policy := health.RetryPolicy{  
    MaxRetries:    5,  
    Delay:         2 * time.Second,  
    BackoffFactor: 1.5,  
}
```

Lesson 4.3: Integration with Boot

- Required vs optional services
- Health check failures
- Graceful degradation

Lab 4: Health Checking

- Implement TCP health check
 - Add retry policy
 - Handle failures gracefully
-

Module 5: Boot Manager (40 minutes)

Lesson 5.1: Boot Manager Overview

- High-level orchestration
- Service configuration
- Boot sequence

Lesson 5.2: Service Configuration

```
manager.AddService("postgres", boot.ServiceConfig{
    ComposeFile: "docker-compose.yml",
    HealthCheck: boot.TCPCheck("localhost", 5432),
    Required:    true,
})
```

Lesson 5.3: Boot Sequence

- Discovery phase
- Grouping phase
- Start phase
- Health check phase
- Summary phase

Lab 5: Boot Manager

- Configure multiple services
 - Set up health checks
 - Implement graceful shutdown
-

Module 6: Remote Deployment (50 minutes)

Lesson 6.1: Architecture Overview

- Control node vs worker nodes
- SSH communication
- Compose file distribution

Lesson 6.2: Host Configuration

```
host := remote.RemoteHost{
    Name:    "production",
    Address: "prod.example.com",
    User:    "deploy",
```

```
    Runtime: "podman",  
}
```

Lesson 6.3: SSH Executor

- Connection pooling
- ControlMaster optimization
- Key authentication

Lesson 6.4: Compose Detection

- Priority order
- Podman-compose vs docker-compose
- Fallback mechanisms

Lesson 6.5: Remote Compose Orchestrator

```
orch := remote.NewRemoteComposeOrchestrator(host, executor, nil)  
orch.Up(ctx, project)
```

Lab 6: Remote Deployment

- Configure remote host
 - Deploy compose project remotely
 - Monitor remote services
-

Module 7: Scheduling Strategies (30 minutes)

Lesson 7.1: Scheduler Overview

- 5 available strategies
- When to use each

Lesson 7.2: Resource-Aware Scheduling

- CPU/Memory/Disk scoring
- Host probing
- Dynamic scoring

Lesson 7.3: Other Strategies

- Round Robin
- Affinity (labels)
- Spread

- Bin Pack

Lab 7: Multi-Host Deployment

- Configure multiple hosts
 - Use affinity scheduling
 - Distribute workloads
-

Module 8: Advanced Topics (40 minutes)

Lesson 8.1: Lifecycle Management

- Lazy boot
- Idle shutdown
- Semaphore control

Lesson 8.2: Event System

- EventBus
- Subscribing to events
- Filtering events

Lesson 8.3: Resource Monitoring

- Container metrics
- System metrics
- Threshold alerts

Lesson 8.4: Service Discovery

- TCP scanning
- DNS discovery
- mDNS discovery

Lab 8: Building an Operator

- Combine all concepts
 - Build a complete operator
 - Handle all edge cases
-

Module 9: Troubleshooting (20 minutes)

Lesson 9.1: Common Issues

- "no compose command found"
- "http+docker" URL scheme error
- Connection refused
- Build context issues

Lesson 9.2: Debug Logging

```
logger.SetLevel(logging.DebugLevel)
```

Lesson 9.3: Performance Tuning

- ControlMaster settings
 - Connection pooling
 - Parallel operations
-

Module 10: Best Practices (20 minutes)

Lesson 10.1: Production Checklist

- Health checks for all services
- Required vs optional marking
- Resource labels
- Monitoring setup

Lesson 10.2: Security Considerations

- SSH key management
- Least privilege
- Network isolation

Lesson 10.3: Scaling Strategies

- Multi-host distribution
 - Resource-aware scheduling
 - Horizontal scaling
-

Resources

Documentation

- [USER_GUIDE.md](#)
- [ARCHITECTURE.md](#)
- [API_REFERENCE.md](#)
- [REMOTE_DEPLOYMENT.md](#)

Code Examples

- [examples/](#)
- [tests/integration/](#)
- [challenges/](#)

Community

- [GitHub Issues](#)
 - [Discussions](#)
 - [Contributing Guide](#)
-

Certificate Requirements

To earn the course certificate:

1. Complete all video lessons
 2. Pass all lab exercises
 3. Submit final project:
 - Build a container orchestrator
 - Support local and remote deployment
 - Implement health checking
 - Use at least 2 scheduling strategies
-

Last Updated: February 2026